

融合Hough变换和卡尺聚类的圆检测算法

钟靖¹, 叶志前^{2, 3}, 吴健^{2, 4} (通信作者)

1. 浙江欧威科技有限公司, 浙江温州, 325000; 2. 浙江大学温州研究院, 浙江温州, 325000

3. 浙江大学生物医学工程与仪器科学学院, 浙江温州, 325000; 4. 浙江大学公共管理学院, 浙江温州, 325000

摘要: 为了提升圆检测算法的准确率, 本文提出了一种利用Hough变换和卡尺聚类的亚像素级圆检测算法。首先通过快速Hough变换对圆进行粗定位, 对粗圆建立卡尺模型逐角度寻找最佳边缘点, 并通过密度聚类排除虚假圆心, 最后渐进式拟合圆, 实现圆心精确定位。实验在本文准备的两个数据集NORM、HARD上与两类图像处理函数库Halcon、Opencv中的圆检测算法进行了比较, 在NORM数据集中, 相比于性能第二的算法, 本算法在准确率持平的前提下, 平均处理时长缩短了79%; 在HARD数据集中, 相比于性能第二的算法, 本算法准确率提高了9.8%, 平均处理时长缩短了87.9%; 实验结果证明本文提出的融合算法显著提高了精度, 缩短了检测时间。

关键词: 圆检测; Hough变换; 卡尺聚类; 密度聚类

中图分类号: TP391.41

文献标志码: A

DOI: 10.19772/j.cnki.2096-4455.2023.10.019

0 引言

随着自动化技术的不断发展, 机器视觉在工业生产中发挥越来越重要的作用, 在半导体、纺织等需要自动对位的领域, 高速、高精度的圆Mark点检测算法起着关键作用^[1-2]。近些年, 面市的基于光学原理的工业检测设备, 例如自动对位曝光机(AAE)、直接成像机(DI)、自动光学检测(AOI)等, 对于圆的检测精度有着很高的要求。在图像处理中, 如何提高对位圆mark点的对位精度和速度是一个难题。本文提出了一种结合Hough变换和卡尺聚类的亚像素级圆检测算法: ①利用Hough变换初步确定圆心位置和半径; ②使用卡尺聚类的方法去除干扰点, 实现圆的精确定位。

1 算法描述

1.1 快速Hough变换

1.1.1 算法概述

Hough变换对图像的检测不涉及灰度阈值的

选定, 对噪声的抗干扰能力强。它将图像空间转换到参数空间, 把所有可能落在边缘上的点进行统计计算, 根据对数据的统计结果确定属于边缘的程度^[3]。圆的笛卡尔坐标表达如式(1):

$$(x-a)^2 + (y-b)^2 = r^2 \quad (1)$$

本算法预先确定圆的目标半径, 使空间从三维降至二维, 有效地减少变换的时间; 计算边缘点的梯度方向; 投影出可能的圆心点坐标; 将重复次数最多的投影点作为圆心。最后利用质心的思想对Hough变换得到的圆心进行二次矫正定位; 使得圆心的位置更准确。

1.1.2 图像预处理与梯度方向计算

Hough变换需要获取图像的边缘信息, 边缘信息由一阶导算子或二阶导算子对灰度图进行卷积获得。本算法使用Sobel算子对图像进行边缘提取。Sobel算子是经典的一阶导算子, 许多工作中都应用了Sobel算子。

计算边缘点的梯度方向的正切值, 计算公式如下。

沿横轴方向:

基金项目: 本文系 2022 年温州市重大科技创新攻关项目“揭榜挂帅·全球引才”专项《基于 PCB-AOI 骨架测量算法的缺陷检测系统开发与应用研究》(项目编号: ZR2022002)

作者简介: 钟靖, 男, 汉族, 湖南长沙, 硕士, 高级工程师, 研究方向: 图像分析及处理; 叶志前, 男, 汉族, 浙江温州, 硕士, 副教授, 研究方向: 医学信息处理。

通信作者: 吴健, 男, 汉族, 浙江杭州, 博士, 教授, 研究方向: 医学人工智能、服务计算和数据挖掘。

$$s_x = \frac{v_x}{\sqrt{v_x^2 + v_y^2}} + 0.5 \quad (2)$$

沿纵轴方向:

$$s_y = \frac{v_y}{\sqrt{v_x^2 + v_y^2}} + 0.5 \quad (3)$$

式中, v_x 为沿x轴方向上的梯度, v_y 为沿y轴方向上的梯度。

1.1.3 圆心投影

根据梯度方向和目标半径可以投影出圆心坐标, 对正方向投影得:

$$x_p = x + r_s \times (s_x + 0.5) \quad (4)$$

$$y_p = y + r_s \times (s_y + 0.5) \quad (5)$$

对反方向投影得:

$$x_{rp} = x - r_s \times (s_x + 0.5) \quad (6)$$

$$y_{rp} = y - r_s \times (s_y + 0.5) \quad (7)$$

式中, (x_p, y_p) 为正向投影后圆心的坐标, (x_{rp}, y_{rp}) 为反向投影后圆心的坐标, r_s 代表搜索半径。当投影的圆心坐标位于规定内, 则该坐标的出现次数累加, 直至所有点被遍历完毕。考虑到因圆孔位于FOV外导致边界出现不完整圆轮廓的情况, 本算法将边界向外拓展了一部分。

1.1.4 圆心坐标矫正

本算法以出现次数最多的点坐标 (x_m, y_m) 为中心, 利用质心思想, 将搜索范围内的其余点坐标用于圆心的坐标矫正, 矫正公式为:

$$x_h = \frac{\sum_{i=-u}^u \sum_{j=-u}^u (x_m + i) \times P_{y_m+j, x_m+i}}{\sum_{i=-u}^u \sum_{j=-u}^u P_{y_m+j, x_m+i}} \quad (8)$$

$$y_h = \frac{\sum_{i=-u}^u \sum_{j=-u}^u (y_m + j) \times P_{y_m+j, x_m+i}}{\sum_{i=-u}^u \sum_{j=-u}^u P_{y_m+j, x_m+i}} \quad (9)$$

式中, u 为坐标最大矫正范围, P 为记录坐标点出现次数的矩阵, 为了提高算法的稳定性, 出现次数较少或越过规定范围的坐标点不纳入计算。最后, 去除圆点附近的干扰点, 得到圆心 (x_h, y_h) , 至此, Hough变换检测结束。

1.2 卡尺聚类

1.2.1 算法概述

卡尺聚类以Hough变换求得的圆心及圆径为

基准, 将圆周均分为若干部分, 每一部分都有对应的角度, 令此角度下的圆边缘点可沿当前角度前后移动一段距离(卡尺长度), 以最大梯度或以某一梯度阈值作为判断标准求出每个角度下的最佳边缘点。求出最佳边缘点后, 以点与相邻点所成圆心角的角度为标准过滤边缘点。对筛选后的卡尺点进行密度聚类, 通过渐进式拟合的方法对最大的类进行拟合, 从而得到目标圆。

1.2.2 最佳边缘点检测

卡尺角度的定义式为:

$$\alpha = \frac{2\pi}{z} \times k + \beta \quad (10)$$

式中, z 为卡尺总数, k 为当前卡尺的编号, β 为初始角度。

根据当前角度计算投影后的 x 、 y 轴上的步长:

$$s_y = \sin\left(\alpha - \frac{\pi}{2}\right) \quad (11)$$

$$s_x = \cos\left(\alpha - \frac{\pi}{2}\right) \quad (12)$$

式中, s_y 为 y 轴方向单位长度在该角度下的投影长度, s_x 为 x 轴方向单位长度在该角度下的投影长度。

点坐标在 α 角度下的动态表述如下:

$$r_c = r_h + r_{rnd}, r_{rnd} \in \left[-\frac{w}{2}, \frac{w}{2}\right] \quad (13)$$

$$x_l = r_h \times \cos \alpha + x_h \quad (14)$$

$$y_l = r_h \times \sin \alpha + y_h \quad (15)$$

式中, (x_l, y_l) 为投影在 α 角度上的边缘点坐标, r_h 为Hough变换求得的半径, r_{rnd} 为动态长度, 其范围由移动路径总长(卡尺长度) w 确定。

利用双线性插值法求该点灰度, 计算横坐标与边界的距离:

$$O_x = \lceil x_l \rceil - x_l \quad (16)$$

式中, O_x 为当前点横坐标与边界的距离。

计算纵坐标与边界的距离:

$$O_y = y_l - \lfloor y_l \rfloor \quad (17)$$

式中, O_y 为当前点纵坐标与边界的距离。

计算横向插值灰度:

$$g_x = g_{\lfloor x_l \rfloor + 1, \lfloor y_l \rfloor} \times (1 - O_{x_l}) + g_{\lfloor x_l \rfloor, \lfloor y_l \rfloor + 1} \times O_{x_l} \quad (18)$$

式中, g 为图像的灰度矩阵, g_x 为点 (x_l, y_l) 横向灰度。

计算纵向插值灰度:

$$g_y = g_{\lfloor x_l \rfloor, \lfloor y_l \rfloor + 1} \times (1 - O_{y_l}) + g_{\lfloor x_l \rfloor + 1, \lfloor y_l \rfloor + 1} \times O_{y_l} \quad (19)$$

式中, g_y 为点 (x_l, y_l) 纵向灰度。

加权合并求得插值灰度 g_{x_l, y_l} :

$$g_{x_l, y_l} = (1 - O_{y_l}) \times g_x + O_{y_l} \times g_y \quad (20)$$

在计算双线性灰度插值时需要注意是否越界, 本算法考虑到边界可能出现圆的一部分轮廓, 将界限向外拓展了一定长度。

以当前点所在延伸方向上的相邻点的双线性插值灰度来计算该点的梯度, 计算前点的投影坐标, 设当前点坐标为 (x_l, y_l) , 下标为 t , 则有:

$$x_l^f = x_l + t \times s_x \quad (21)$$

$$y_l^f = y_l + t \times s_y \quad (22)$$

计算后点的投影坐标:

$$x_l^b = x_l - t \times s_x \quad (23)$$

$$y_l^b = y_l - t \times s_y \quad (24)$$

求取沿途点灰度并累加, 得到平均灰度值 $\bar{g}$:

$$\bar{g} = \sum_{j=1}^w \frac{g_{x_l^b, y_l^b} + g_{x_l^f, y_l^f}}{w + 1} \quad (25)$$

坐标点沿着所属卡尺的方向从外向内 (或从内向外) 步进, 同时计算沿途点的梯度。

以均值滤波去除梯度噪声, 滤波公式如下:

$$u_i = \frac{u_{i+1} - u_{i-1}}{2}, i \geq 1 \quad (26)$$

式中, u 代表沿途点梯度的集合。

考虑到未来更加复杂的情况, 本式可拓展到计算相邻若干点的梯度, 也可通过高斯分布的方式计算梯度。下一步需要找出梯度 (绝对值) 最大的点, 由该点重新计算实际半径和相应的投影点。列式计算半径 r_f 为:

$$r_f = r_h - w + g_m \quad (27)$$

式中, g_m 为此方向上的最大梯度值。

$$x_f = r_f \times \cos \alpha + x_h \quad (28)$$

$$y_f = r_f \times \sin \alpha + y_h \quad (29)$$

至此, 便求得了角度为 α 时的最佳边缘点, 计算所有卡尺的最佳边缘点并纳入集合。

1.2.3 干扰点过滤

受图像噪声干扰, 并非所有方向上的最佳边缘点都是真实的圆孔边缘, 因此需要一种快速有效的方法筛选这些卡尺中的无关项, 本算法通过计算相邻卡尺点所成圆心角的度数的方式去除干扰点, 投影中心点, 根据所成圆心角度数过滤。

最小角度阈值 T 定义为:

$$T = (\varepsilon / z - \pi) \times b \quad (30)$$

式中, ε 为最大角度, b 为可调整参数。

取点集中下标为 i 的点 p_i , 构成投影点与前、后两点的向量 $\vec{v}_1$ 、 $\vec{v}_2$:

$$\vec{v}_1 = (x_{i-1} - x_i, y_{i-1} - y_i) \quad (31)$$

$$\vec{v}_2 = (x_{i+1} - x_i, y_{i+1} - y_i) \quad (32)$$

计算圆心角度数, 由于向量长度均为 1, 正弦值、余弦值的计算简化如下:

$$\sin \varphi = \vec{v}_1 \cdot x \times \vec{v}_2 \cdot y - \vec{v}_2 \cdot x \times \vec{v}_1 \cdot y \quad (33)$$

$$\cos \varphi = \vec{v}_1 \cdot x \times \vec{v}_2 \cdot x + \vec{v}_1 \cdot y \times \vec{v}_2 \cdot y \quad (34)$$

式中, φ 为待求圆心角的角度。

转化为正切值并求出 φ :

$$\varphi = \arctan \left(\frac{\sin \varphi}{\cos \varphi} \right) \quad (35)$$

将 φ 与最小角度阈值 T 进行比较, 若小于, 说明该点是干扰点, 舍去并计算下一点的圆心角。

若大于, 继续计算该点与相邻前置点所成角度 γ :

$$\gamma = \arctan \left(\frac{y_{i-1} - y_i}{x_{i-1} - x_i} \right) + 0.5 \times \alpha \quad (36)$$

用半径长度和弧度构造极坐标向量:

$$\vec{v}_3 = (r_f \times \sin \gamma, r_f \times \cos \gamma) \quad (37)$$

矫正最佳边缘点 p_i :

$$p_i = (x_f + \vec{v}_3 \cdot x, y_f + \vec{v}_3 \cdot y) \quad (38)$$

计算校正后与旧点之间的偏移, 舍去处于边界或对比度较低的点。保存偏移不超过最大偏移量的组合 (p_{i-1}, p_i, p_{i+1}) , 重复上述步骤, 完成卡尺点的筛选。

1.2.4 密度聚类及渐进式圆拟合

本文使用经典的DBSCAN聚类法对边缘点进行聚类。为了确保拟合的准确性,若最大聚类内元素少于卡尺总数的六分之一,则视为无效聚类。本文以3度为标准划分出共120个卡尺,此时若最大聚类中的元素小于20则舍去,在20和30之间则根据对比度判断取舍。得到最大聚类后,分为两种模式拟合圆。在非圆心稳定优先模式下,计算类簇中卡尺的最佳边缘点与圆心的距离,计算此距离与Hough圆径的偏差,以此为排序基准降序排序,滤除干扰卡尺。

为保证有足够数量的点参与拟合,同时滤除干扰卡尺,本算法规定,在卡尺角度小于90度时直接纳入拟合集,角度在每一次渐进拟合后按步长增大,当角度不超过90度时直接将该点纳入集合,角度大于90度时,计算该点半径与原半径的偏差,若偏差超过偏差阈值,则纳入拟合集中。偏差阈值 ρ 定义为:

$$\rho = \frac{2\pi - \min(\pi, \theta)}{\pi} \times \lambda \quad (39)$$

式中, θ 为当前角度, λ 为可调节参数。

在圆心稳定优先模式下,直接对类内所有最佳边缘点采用渐进式拟合的方法拟合圆,需要注意的是,当类内点数明显较少时,应判定为拟合失败。

完成聚类分析后,对聚类内卡尺进行圆拟合,对圆拟合点拟合圆的常用方法包括平均值法、加权平均法和最小二乘法等,这些方法被广泛地应用于工业生产当中,本算法采用渐进式拟合法拟合圆,根据条件判断是否将点纳入拟合集中,渐进式拟合可以看作是若干次最小二乘法的组合,渐进式拟合的计算过程如下:

$$a_1 = z \times \sum_{i=1}^z x_i^2 - \left(\sum_{i=1}^z x_i \right)^2 \quad (40)$$

$$b_1 = z \times \sum_{i=1}^z y_i^2 - \left(\sum_{i=1}^z y_i \right)^2 \quad (41)$$

$$a_2 = z \times \sum_{i=1}^z x_i^3 + z \times \sum_{i=1}^z x_i y_i^2 - \left(\sum_{i=1}^z x_i^2 + \sum_{i=1}^z y_i^2 \right) \times \sum_{i=1}^z x_i \quad (42)$$

$$b_2 = z \times \sum_{i=1}^z y_i^3 + z \times \sum_{i=1}^z x_i^2 y_i - \left(\sum_{i=1}^z x_i^2 + \sum_{i=1}^z y_i^2 \right) \times \sum_{i=1}^z y_i \quad (43)$$

$$d = z \times \sum_{i=1}^z x_i y_i - \sum_{i=1}^z x_i \times \sum_{i=1}^z y_i \quad (44)$$

$$x_{out} = \frac{a_2 \times b_1 - b_2 \times d}{2 \times (a_1 \times b_1 - d^2)} \quad (45)$$

$$q = 2 \left(x_{out} \times \sum_{i=1}^z x_i + y_{out} \times \sum_{i=1}^z y_i \right) - \sum_{i=1}^z x_i^2 + \sum_{i=1}^z y_i^2 \quad (46)$$

$$y_{out} = \frac{b_2 \times a_1 - a_2 \times u}{2 \times (a_1 \times b_1 - u^2)} \quad (47)$$

$$r_{out} = \sqrt{x_{out}^2 + y_{out}^2 - \frac{q}{z}} \quad (48)$$

式中, (x_i, y_i) 为第 i 个卡尺点的坐标, (x_{out}, y_{out}) 为拟合圆的坐标, r_{out} 为拟合圆的半径。相比其他拟合方式,渐进式圆拟合通过逐渐增加点并多次拟合圆的方式,避免了重复运算,且内部不保留点集,减少了内存占用。圆拟合变换得到的圆即为算法的最终输出。

2 实验结果与分析

2.1 算法验证

我们采集了实际作业中的图像作为实验数据集,数据集分为检测成功数据集(命名为NORM,图像尺寸为1288*964,共2400张)和检测失败数据集(命名为HARD,图像尺寸为1288*964,共698张)。其中设置HARD数据集的目的是测试改进后的算法在非理想图片上的检测性能,而设置NORM数据集的目的是测试改进算法是否能在理想图片下保持高准度和低处理

时长。在实验中,本文分别测试了快速Hough变换算法和卡尺聚类算法的处理效果,处理结果如图1所示。其中,(b)为快速Hough变换算法单独作用时的处理结果,可以看到左半部分的轮廓未能很好地拟合真实轮廓,出现了较大偏差,证明单一的Hough变换在复杂条件下的处理精度有限,因此引入本文提出的卡尺聚类算法对轮廓进行精确调整,如(c)所示,十字标记为卡尺法求得各方向上的最适边缘点,右侧轮廓紧密贴合于实际边缘上,与(b)中保持一致;左侧轮廓则向内侧真实边缘靠拢,与实际边缘的贴合度要远优于(b)中处理效果。图中(d)为快速Hough变换结合卡尺聚类的最终处理结果。显然,与单一Hough变换相比,本融合算法能够有效矫正Hough变换所得轮廓中的不准确轮廓部分,将错误偏移的轮廓向实际边缘拉拢,检测出的轮廓的精确度相比(b)有了明显的提升,提升效果达到了理论中的预期^[4]。

融合算法在不同情况下的检测效果如图2所示,(c)(d)(g)是检测失败样本中常见的类型,占比较大,现有的算法不能很好地处理这些

样本,本融合算法能够准确地检测出这些图样中的圆,解决了检测时遇到的主要问题,(a)(b)(h)在检测失败的样本中占比小,噪声干扰严重,检测难度要高于前者,本算法将像素点坐标精确到亚像素级,能够精确定位出位置,解决了检测时遇见的次要问题。部分圆孔因未能完全出现在相机FOV中产生了(e)(f)图像,本算法拓展了投影边界,允许圆心能够投影在边界之外来解决这个问题。图2证明本算法能够在上述不理想的图样中准确检测到圆的轮廓,能够从不连续边缘中检测出圆轮廓,具有良好的泛化性。

2.2 算法对比

本文选择Halcon(机器视觉开发工具包)中改进的圆检测算法和开源图像处理库OpenCV中改进的圆检测算法作为对照组与本文提出的融合算法进行对比。并从算法的平均处理时长(AVT)和准确率(ACC)来评估算法的实际效果。将数据集NORM均分为3份,将数据集HARD均分为4份,进行测试。本算法在NORM和HARD数据集中的准确率均为100%。对于HARD数据集,虽然改进Halcon算法的消耗时间最短,但此

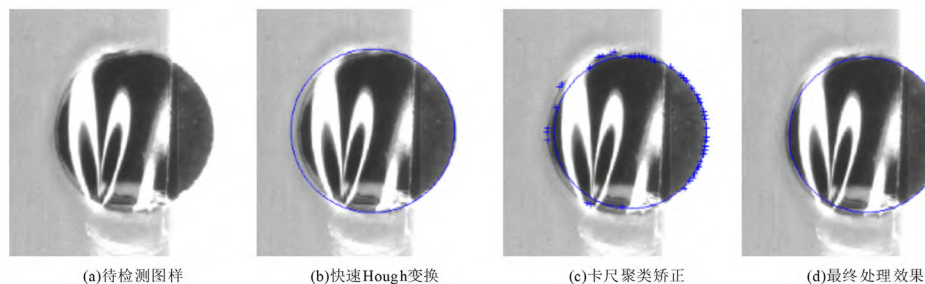


图1 Hough检测和卡尺聚类独立检测及融合检测效果

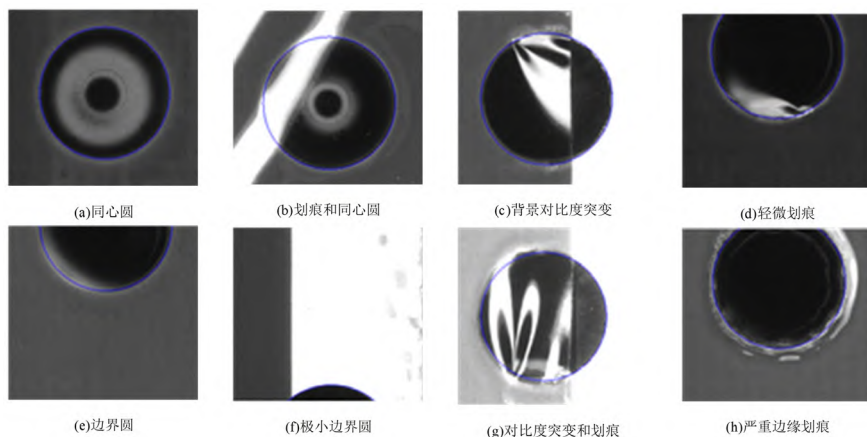


图2 融合算法在不同情况下的检测效果

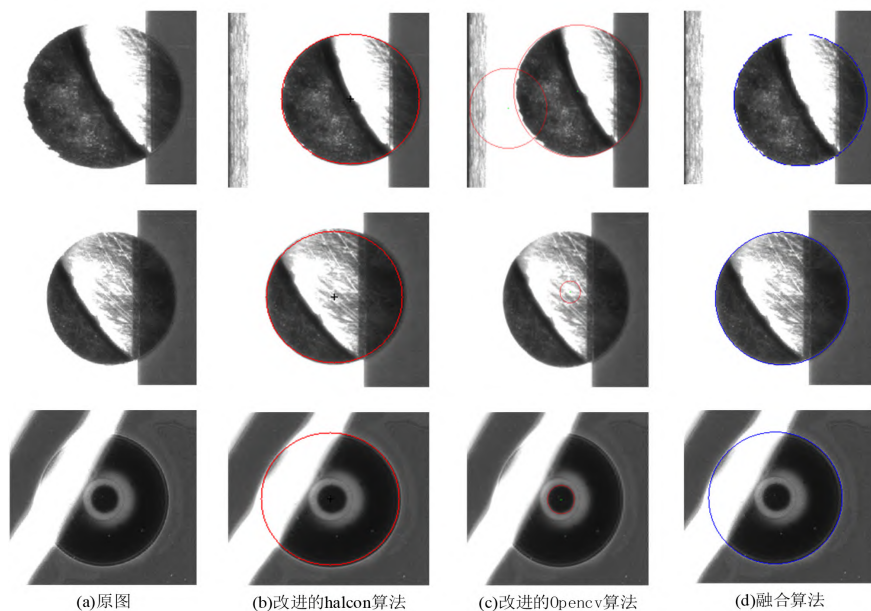


图3 三种算法的圆检测效果

算法的准确率过低,准确率不足60%,无法满足实际检测目标,改进OpenCV算法的准确率和稳定性仅次于本算法,但此算法的平均处理时间过长,在NORM和HARD数据集上的平均处理时间分别是本算法的4.7和10.2倍,时间效率过低。相较而言,本算法的综合表现要优于前两者。图3为三种算法的检测结果。由图可得,OpenCV算法在这三类图样中的检测结果十分不理想,改进Halcon算法的定位与实际轮廓仍然存在一定偏差。对于这些图样,本算法检出的轮廓与实际边缘最为吻合,检测精度要高于前两种算法^[5]。

3 结论

本文提出了一种亚像素级精度下的结合Hough变换与卡尺聚类的融合算法来提升圆检测的准确性,本检测算法无需设置灰度阈值,对噪声具有较强抗干扰能力。此外,本算法将检测精

度提升到了亚像素级,通过双线性插值法计算灰度值来提升检测效果。实验结果证明,本算法具有理想的处理速度,有效提升了圆mark点检测精度。算法的实际准确率和运行速度要优于现有的两种图像处理函数库Halcon及OpenCV中的检测算法,能够应用于工业检测。

参考文献

- [1] 袁坤,彭和平.基于曲线拟合的随机圆检测算法[J].软件导刊,2022,21(11):52-57.
- [2] 庞艳丽.基于共形几何代数的圆检测算法研究[D].天津:中国民航大学,2022.
- [3] 李建森,项偲.基于随机采样的随机Hough变换快速圆检测算法[J].科技创新与应用,2021,11(29):128-130.
- [4] 龚昕,张楠.基于Hough变换的圆检测算法的改进[J].信息技术,2020,44(06):89-93+98.
- [5] 朱正伟,宋文浩,焦竹青,等.基于随机Hough变换改进的快速圆检测算法[J].计算机工程与设计,2018,39(07):1978-1983.